

The roster is not admin work. It is the machine that makes the store work — and the page sells it as a coupon.

A roster tells the store which swimmers exist, which group they are in, what gear that group requires, what sizes to collect and who to remind before a deadline. Membership and the 35% code are byproducts of that job being done: we earn accounts because a team got outfitted, not the other way round. The page states the byproduct and never states the function.

WHAT YOUR DATA CORRECTED

A large share of first attempts *fail*, there is a multi-day gap between store created and roster live, and roster issues are a top support driver. Admins are trying and losing — a different problem from admins not being persuaded. So upload repair ranks first here, and the copy inversion became a cheap fast-follow rather than the headline.

Scope: Dashboard and Roster, deliberately · August 2026 · companion prototype covers the near-term column end to end

PART 1

Discovery plan

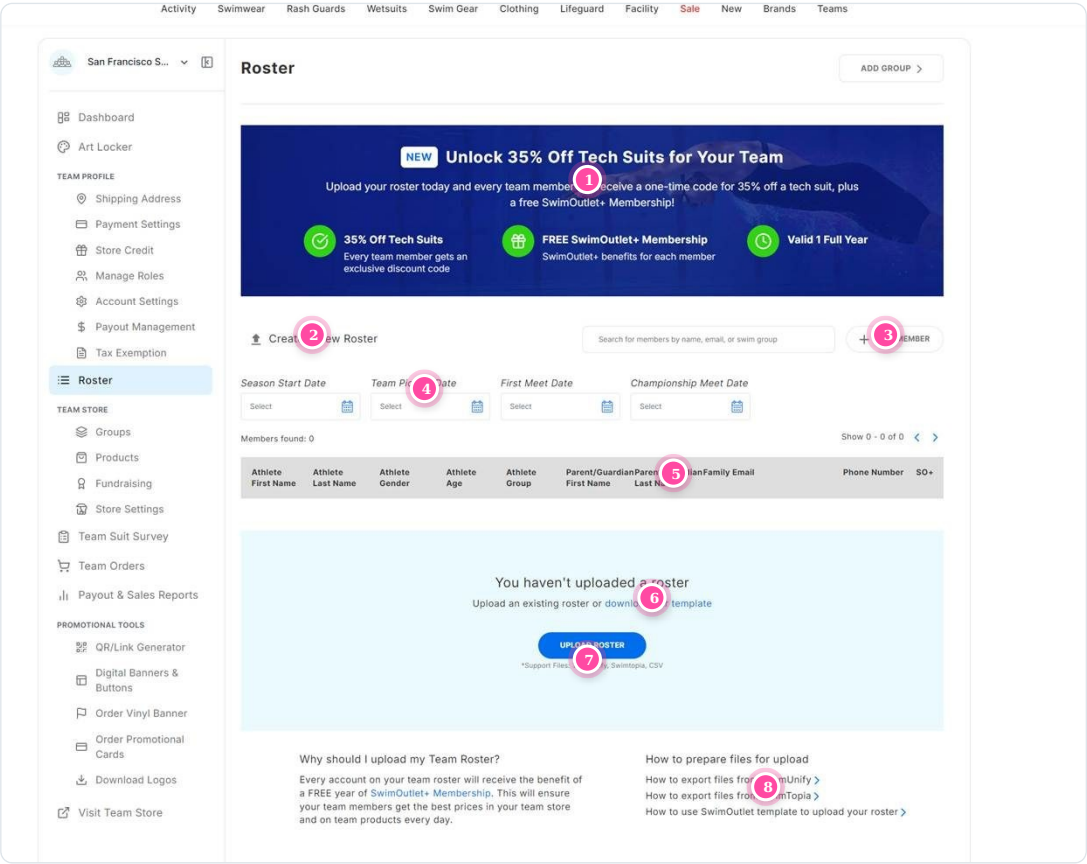
Six questions in the order I would answer them. The first four need no one's calendar, which is why they come first.

QUESTION	METHOD	WHO / WHERE	WHAT I WOULD LOOK FOR
Where exactly does the funnel break: store created → roster uploaded → invite sent → first family order?	Data pull	BI / analytics owner	Step-level drop-off, and the distribution behind that multi-day median. A long tail of teams that never come back matters more than the median.
What is failing validation on the first attempt, ranked?	Data pull on validation events — if untracked, this becomes work item zero	Engineering, current logs	Top five error classes by frequency and by teams affected, and whether they cluster by source system. Clustering means fix parsing, not UI.
What files are admins actually bringing?	Audit 30 real attachments from the last two seasons	Support queue: TeamUnify, SwimTopia, Hy-Tek, SwimCloud, SportsEngine, parent Sheets	Header variance, name formats, date-of-birth vs age, sibling email reuse. This produces the alias dictionary the parser needs and doubles as a test set.
Where do admins abandon, and what does support fix by hand?	Session replay plus ticket review and two agent interviews	Roster sessions for new stores; team sales support agents	Repeated attempts with the same file, dead clicks on help links, whether anyone scrolls below the fold — and every manual edit an agent makes, since each is a candidate for automation.
Who is the admin, and when in the season does she do this?	6–8 interviews, 30 minutes	3 coaches on different source systems, 2 volunteer parents, 1 admin of a 200+ swimmer team, 2 regional reps	Who physically holds the data, whether she would delegate collection, and what else is happening the week we ask.
What is a roster worth, and are we already testing the copy?	Data pull, then ask	Finance / BI; growth; partnerships	Repeat rate, AOV, SwimOutlet+ activation and code redemption for roster-sourced accounts vs organic — this sets the ceiling on what the fix is worth. Then: is the banner copy tested or shipped, and is roster data in scope of the existing SwimTopia partnership?

PART 2 A

The page, as an admin with a TeamUnify export sees it

Eight findings on the live page. Click a pin or a card. Pink means the same thing here as in the prototype: something I would change.



1 INVERTED

The pitch is inverted

The banner sells a membership tier and a discount code. It never says what a roster does. Membership is what we earn once the store can do its job.

HOW I'D VALIDATE IT

Ask growth whether this copy is tested or shipped, then A/B job-first against perk-first on the same banner.

2 CONFUSING

"Create a New Roster" has no stated purpose

A team with no roster is offered a second one, and the obvious shortcut for a returning team — last season's roster — is not offered at all.

HOW I'D VALIDATE IT

Session replay for new stores: how many click it, and what they do next.

3 MISSING

Add Member accepts a member who can never be reached

The modal saves a member with no email, silently creating a row that cannot receive the benefit the banner just promised. Nothing marks the field as required.

HOW I'D VALIDATE IT

Share of existing roster rows with no family email; tickets reading "my families never got the code".

4 CONFUSING

Four date fields, no stated purpose or requirement

No indication of whether they block the upload, what they drive downstream, or what to do when the schedule is not set yet in August.

HOW I'D VALIDATE IT

Three coach interviews: is the season calendar even known at store setup?

5 CONFUSING

Three surfaces, three vocabularies, no required marks

The grid says Athlete First Name and Family Email; the template says Account First Name and Email; Add Member asks for Swimmer Name; the help article uses a fourth set. An admin who hits an error has no stable term to search on.

HOW I'D VALIDATE IT
Support ticket review: which field names admins use to describe what broke.

6 MISSING

No mapping, no preview, no partial success

If headers do not match, the only path is reshaping the file by hand in Excel. Nothing shows what we understood before committing, and nothing says 300 of 312 rows were fine.

HOW I'D VALIDATE IT
Validation failures by class, plus the 30-file audit.

7 MISSING

Only two of six source systems are addressed

Teams on Hy-Tek, SwimCloud, SportsEngine and parent-maintained Sheets are the population most likely to fail, and the page speaks only to the two exports that already work.

HOW I'D VALIDATE IT
Attachment audit segmented by source system.

8 CONFUSING

The guidance sits after the moment it is needed

The three how-to guides are below the upload button and below the fold. An admin who needed to prepare her file differently learns that after she has already failed.

HOW I'D VALIDATE IT
Scroll depth on the Roster page and click-through on the guide links.

What works, and is worth keeping

- The empty state is honest, offers two paths rather than one, and names TeamUnify and SwimTopia explicitly. An admin holding one of those exports learns in a glance that she is in the right place.
- The benefit is stated where the work is asked for, not buried in a help centre — the framing is wrong, but the placement is right.
- Add Member exists, so a one-swimmer correction never needs a file.

PART 2B

What validation will choke on, and what to do about each

Note on inputs: I worked from the SwimOutlet template and from what exports out of the systems named in the brief actually look like; these are the classes I would confirm first against roster_sample.csv. Even the clean template contains three of them — a trailing space in an athlete first name, an apostrophe inside an email address, and a parent surname spelled differently from the athlete surname in the same row.

The rule I applied. Silently fix only what has exactly one correct answer. Fix and confirm anything where I am changing meaning. Flag anything only a human can decide. Hard-reject only when no row can be formed at all, or when importing would breach policy. And never make the admin start over: fixing happens in place.

PROBLEM CLASS	EXAMPLE	DISPOSITION	LOGIC
Header names differ from our template	First Name, Parent1 Email, Sex, League Age	b · fix + confirm	Confidence is high, but I am asserting meaning — so she confirms once, and we remember it next season.
Unknown extra columns	USA-S ID, Hy-Tek athlete ID, notes	a · silently ignore	Lossless. Listed as ignored in the summary so nothing feels swallowed.
Formatting, structural and	trailing spaces; blank	a · silent fix	One correct answer in every case. We cannot make

encoding noise	rows; a "Total: 42" row; BOM; semicolons; .xlsx renamed .csv		vendors change their exports, so tolerance is our job. The summary still reports 42 read, 2 skipped — the count is what she checks.
Values in a shape we do not expect	merged "Sturgeon, Nicola"; date of birth instead of age; gender as M / F / 1 / 2 / blank	b · fix + confirm	All three are derivable and all three change meaning, so all three get shown once rather than assumed. Which half of a merged name is the surname is a guess; league age depends on a cutoff date, and a wrong age is a wrong suit size; mapping M to Male is safe, but a blank gender stays blank — guessing a swimmer's gender to complete a row is not a trade I would make.
Phone mangled by Excel	5.03555E+09; lost leading zero	a · normalise c · flag if unrecoverable	Recoverable formatting is noise; unrecoverable is real loss and only she has the number. Never blocks the import.
Email malformed, unusual, or possibly a minor's	missing @; an apostrophe inside the address; first-initial-surname at a school domain	c · flag to resolve	The one required field that silently decides whether a family gets the benefit, so a likely typo gets a look, not an auto-edit. Policy forbids listing addresses for under-18s, and only she knows whose address it is.
Required value missing	blank family email on 12 of 312 rows	c · flag, import the rest	Not an error — missing information. Quarantine those rows, commit the other 300.
Duplicate swimmers vs siblings sharing an email	the same athlete twice; two athletes, one parent address	b · confirm duplicates a · accept siblings	Two swimmers in one club can share a name, so the match is a judgement. Siblings sharing a family email is normal and correct — flagging it is the fastest way to lose a large club's trust. Never dedupe on email alone.
Nothing parseable	no header row, zero data rows, corrupt file	d · hard reject	The only case with no partial value to preserve. Reject in plain language with a sample file, never a code.

PART 3

Target-state upload experience

Three steps, always the same three: **drop the file** → **review what we understood** → **confirm**. The review step is the product; everything else is plumbing. Each scenario below is clickable in the prototype.

First upload, brand-new team

- The page leads with what the roster does for the season, with membership and the code as the second line, and names the three fields we need before she opens anything.
- One drop zone accepting CSV, XLSX and the named platform exports; the template download demoted to a fallback link.
- Parsing happens in the flow, so she lands on the review grid seeing her own swimmers' names — the moment the tool starts to feel trustworthy.
- Commit and invite are separated: nothing reaches families until she chooses, and the benefit clock does not start on a roster she is still editing.

Column mapping when the file does not match

- Auto-map from the alias dictionary built in Part 1, each mapping shown as a best guess with a one-click override and a "not in my file" option.
- Unmapped optional fields proceed; unmapped required fields block the commit and say which field and why.
- Mappings save per team and per detected source, so the second upload never costs what the first did.

Partial failures: 300 good rows, 12 bad

- **Import the 300.** Refusing the whole file over 4% of rows is the behaviour that sends people to support.
- The 12 land in a fix queue on the roster, editable in place, grouped by problem class — twelve missing emails are one task, not twelve.
- A downloadable file of just the failed rows, in the shape we accept, for whoever would rather work in Excel or forward it to a team parent.
- Re-importing is idempotent, so the safest instinct — try again — is safe.

Duplicates and returning-season re-uploads

- Match key: athlete first name, last name and date of birth or age, scoped to the team. Family email is a secondary signal only — siblings share it.
- A re-upload shows a diff before anything commits: **X new, Y changed, Z on the roster but not in this file**. The Z group gets an explicit choice — leave or archive. Never silent deletion.
- Benefits are idempotent per account: no second membership year, no second code, and three re-uploads do not send three emails to the same parent.

Mid-season changes

- Add Member for one swimmer, and a shareable enrolment link for the rest — sent by email or text, families enter their own details. The only design here that removes the file from the coach's desk entirely.
- New members inherit the benefit on join, with status visible per row: enrolled, invited, declined. Today that column cannot distinguish those states.

Where I would draw the v1 line

IN V1 · 8 WEEKS, 2 ENGINEERS, SHARED DESIGNER

- Tolerant parsing: encoding, delimiter, file-type detection
- Alias auto-mapping with confirmation
- The review-and-fix grid
- Partial import, row-level fix queue, downloadable failed rows
- One vocabulary and required-field marking across all three surfaces
- Job-first benefit copy
- Instrumentation on every step

DELIBERATELY NOT IN V1

- The shared enrolment link — collects minors' names on an unauthenticated page, so it needs a privacy review before a design review
- Duplicate-a-previous-roster
- Mapping memory across seasons
- Image ingest for paper rosters
- Platform sync

WHY

Each is defensible later. None is defensible before first-attempt success rate is fixed.

THREE IMPORTS I WOULD BORROW FROM

Stripe. The dry run: nothing commits until you have seen a summary, and failures come back as a file you can fix and resubmit.

Flatfile (and Airtable). Mapping and correction inside a grid in the flow, so the file never reopens in Excel; mappings remembered.

Mailchimp. Dedupe on a stable key, with every automatic decision explained in one plain sentence.

HOW I CHOSE THE ORDER

WSJF, with the season as the deadline that cannot move

It is August. Roster upload clusters at season start, so anything shipped before September catches a full cohort of teams and anything that misses it waits a year for the next natural moment. That single fact outranks elegance, and it is why label work beats the interesting ideas.

#	ITEM	VALUE	TIME CRIT.	RISK RED.	SIZE	WSJF
01	State what the roster does before what it earns the family Job-first copy at the point of the ask	8	13	5	1	26.0
02	One vocabulary + required fields marked across all three intake surfaces Roster grid · CSV template · Add member	8	13	8	2	14.5

03	Inline definitions for jargon and optional fields Fit kits, quotes, the four season dates	3	5	3	1	11.0
04	Duplicate an existing roster Season over season, not a fresh export	5	8	3	2	8.0
05	Upload repair — tolerant parse, editable review, row-level errors Accepts all six named sources as they are	13	13	8	5	6.8
06	Checklist resequenced by dependency, with effort tiers Payout before publish; roster before invite	5	5	3	3	4.3
07	Share link — delegated roster and membership enrolment New unauthenticated surface; privacy review required	13	8	8	8	3.6
08	Image ingest for paper rosters Parked — needs the review grid first, and a count of paper-roster teams	5	2	2	13	0.7

The copy change outranking the engineering is the point, not an artifact of the math.

It is the cheapest item on the board and it sits upstream of everything else: an admin weighing whether this is worth an evening is being sold a \$5 membership rather than a team that shows up to the first meet in the right suits. That is a sequencing error in the pitch, and sequencing errors are free to fix. It is also the score I would most expect the group to argue with — which is what the scores are for.

Three solutions that get discussed as one

A · IN THE NEAR TERM

Tolerant import with an editable review

Solves: *they have the data, in the wrong shape*. Deterministic, cheap, and what the prototype demonstrates.

B · LATER, GATED

Image ingest

Solves: *they have no digital file at all* — a paper sign-up sheet, a screenshot. Different population, different technology, different risk profile.

C · LATER, THE BET

Roster sync with the team-management platform

Solves: *the transfer itself, and the fact that a roster goes stale*. Not a better importer — no import at all.

The bigger swing is cheaper than it sounds

SwimOutlet and SwimTopia have been partnered since January 2022 — teams already embed their SwimOutlet Team Store into their SwimTopia site through a snippet, and that integration has been extended once since. The relationship, the contract and a technical touchpoint exist. This proposal runs the same pipe the other way: store content flows into SwimTopia today, and the roster would flow back out. **That is an amendment and a sync job, not business development from zero.**

Sequencing follows: SwimTopia first, where there is a partner and a shared incentive — their own marketing sells the partnership on saving parent volunteers time, and roster re-entry is the time it has not saved yet. TeamUnify is a separate, harder conversation with no agreement I can find, and bundling the two into one estimate is how this gets mis-sized. **What I would verify first:** whether roster data is already in scope of that agreement. Someone in the room knows.

PART 4

How I would ship it and know whether it worked

Leading, weekly

first-attempt success rate	rows auto-mapped without intervention	abandonment on the review step	median store-created → roster-live
tickets per 100 upload attempts	fix-queue completion		

Lagging, per season

new stores with a live roster in 14 days households activated per store SO+ activation and code redemption time to first family order

repeat rate and AOV vs organic support cost per team

The one I would put on the wall: first-attempt success rate. It is the failure the brief names, it drives the support cost, and it is the only one of these an admin experiences directly.

Instrumentation

Ships in week one on the existing flow, so the redesign has a baseline. Events: upload_started, file_parsed (source guess, rows, columns), mapping_auto (matched, unmatched), mapping_overridden (field), validation_error (class, affected rows), rows_committed, rows_quarantined, fix_applied (class), invite_sent, member_activated. The error class from Part 2b is a first-class dimension, so within a week we can rank causes instead of debating them. Log file shape — header names and counts — not file contents.

Would I A/B test it?

Not the flow. The population is small, seasonal and close to one-shot per team, so there is no path to a powered test inside 8 weeks — and holding half of new teams on a path we know fails, during the exact weeks they onboard, has a real cost in support load and goodwill. Instead: instrument, then pre/post segmented by source platform, a 10% canary watching error and abandonment rates, and five moderated sessions on real files before we ship.

Yes to two narrow tests. The banner — job-first against the current perk-first copy, one variable, a real hypothesis, and the losing variant costs nothing. And the invite email to families, where volume reads a result quickly.

Sequencing: three ships inside the 8-week horizon

The horizon is 8 weeks to first meaningful ship, but nothing here needs to wait that long. Three increments, each usable on its own, each measurable against the week-one baseline.

WEEK	FOCUS	SHIPS
1	Instrument the current flow. Cluster two seasons of roster tickets by error class; generate the alias dictionary and a synthetic test corpus from real file shapes	Baseline for first-attempt success, and a ranked error taxonomy instead of an assumption. Designer starts the grid.
2-3	Tolerant parser: encoding, delimiter and file-type detection; alias auto-mapping; the validation taxonomy from Part 2b	Ship 1 — behind a flag to 10% of new stores. Any named source format parses without hand-reshaping.
4-5	Review-and-fix grid: mapping confirmation, per-row errors grouped by class, in-place editing, partial import, downloadable failed rows	Ship 2 — 300-good-12-bad works end to end; support works the same queue as the admin.
6	One vocabulary across grid, template and Add Member; required-field marking; job-first copy; guidance above the fold	Ship 3 — the fixes that prevent errors rather than catch them. Copy A/B starts.
7-8	Five moderated sessions on real files; widen the canary; measure against the baseline; fix what the sessions surface	Go/hold on evidence rather than the calendar, with two weeks of slack the plan assumes will get used.

Where AI changes this estimate, and where it does not

Two of those weeks look different than they would have two years ago, and one of them is week one. Clustering a season of support tickets into a ranked error taxonomy, proposing an alias dictionary from a few hundred real header rows, and generating a synthetic corpus of malformed files to test the parser against are hours of work rather than days — and the test corpus is normally the slowest part of building tolerant parsing. Front-end scaffolding compresses similarly: the companion prototype, including mapping, validation and partial import, was built in an evening.

What does not compress: the privacy review on a family-facing page collecting minors' names, five moderated sessions with real coaches, QA against real files where a wrong email fails silently, the partnership conversation, and the season itself. Those are why the plan keeps two weeks of slack rather than promising a week-four finish.

The more interesting use is in the product, not the pipeline. A deterministic alias dictionary will never anticipate every header a parent's spreadsheet contains, and a model is a good fallback mapper for exactly that case. The disposition table in Part 2b is already the governance frame for it: **a model may propose, a human confirms, and a model never silently fixes.** Anything a model guessed lands in category (b), never (a) — which is what makes model-assisted mapping safe to ship on a file that decides whether a family ever hears from us.

Risks I would flag on day one

- **Season timing.** Upload clusters at season start. Shipping in September catches this year's cohort; slipping past it means waiting a year. That is why the ordering front-loads parsing over polish.
- **The shared designer.** One grid carries this release. If design capacity slips, weeks 4-5 are the critical path, not the parser.

- **Support workflow.** Agents fix files by hand today. If they cannot work the new fix queue, the tickets do not go away, they change shape. They are users of this release too.
- **What I still do not know.** Whether the cliff is before upload rather than during it. If most teams never reach the Roster page, this plan is aimed at the wrong screen and onboarding takes its place — which is exactly what week one is for.

BEFORE I WOULD FILE ANYTHING

Follow-up questions, and what each answer moves

This board is built to be edited in the room. The scores exist to be argued with, and I would expect at least two rows to move once people with the real numbers weigh in.

QUESTION	WHO'D KNOW	WHAT IT MOVES
Is the enrollment banner copy tested, or shipped?	Growth / whoever owns the surface	Tested → item 01 becomes a variant recommendation, not a change request. Shipped → item 01 stands and instrumenting the surface joins the near-term list.
Is roster data in scope of the existing SwimTopia partnership?	Partnerships / BD	In scope → the sync bet gets much cheaper and moves up. Not in scope → it stays parked and image ingest gets a second look.
How does a filed ticket become sprint work?	The prioritization group	Determines the form this board should take before I bring it — scores, a written case, or both.
Who owns copy on team-store surfaces?	Growth or the team-store line	Changes who I would score item 01 with, and whether it is mine to file at all.
What does the engineering team want in a ticket before they pick it up?	The dev team directly	Cheap to adopt and saves a round-trip across time zones — better asked than assumed.

Scope note. I stayed inside Dashboard and Roster on purpose. I noticed adjacent problems — two undifferentiated shopping paths after signup, and a "share with team leader" action whose purpose is not stated on a conversion-critical screen — and chose not to open them here. Both sit upstream of this brief and deserve a proper look rather than a passing one.

Method: WSJF with Fibonacci scoring — value, time criticality and risk reduction over job size. **Status:** all scores and effort estimates are hypotheses formed from the product surface, its help documentation and public pricing; none are drawn from SwimOutlet's internal data.

Open the clickable prototype · Download this response as a PDF